

SEMINAR PRESENTATION

Self-Edit:

Fault-Aware Code Editor for Code Generation

PRESENTED BY : Dana Nmarny, Hala Hillou

The Problem

02 / 12

Why do LLMs struggle with code generation?

7%

GPT-3 pass@1
on APPS-test

⚠ Syntax Errors

LLMs produce Python 2-style print,
wrong brackets, missing colons

⚡ Runtime Errors

NameError, IndexError, TypeError
crash the program at runtime

✗ Wrong Answers

Logical errors — code runs but
outputs incorrect results

Can we fix AI-generated code using its own execution results
— instead of generating from scratch?

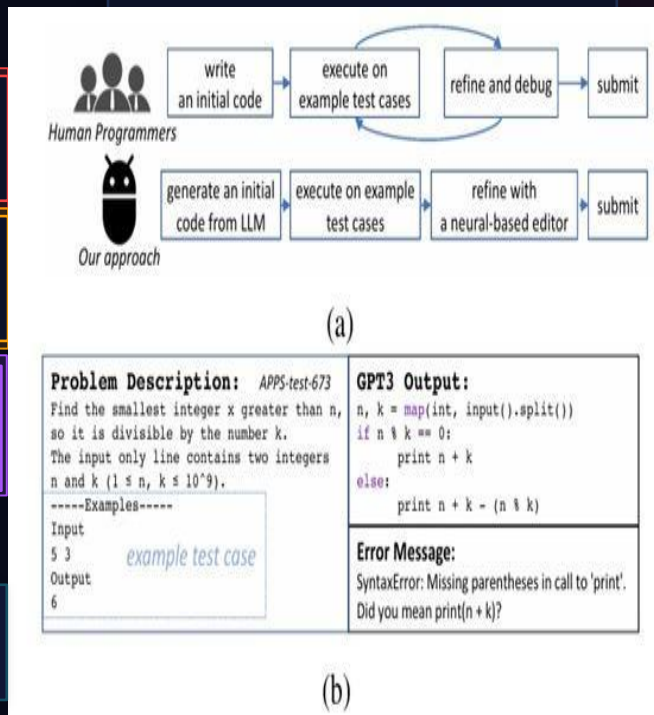


Figure 1 — GPT3 output with SyntaxError on APPS-test

Core Idea

03 / 12

Inspired by how human programmers actually work



Human Programmers

Write



Execute



Debug



Submit



Self-Edit (Our Approach)

Generate



Execute



Edit



Submit

◆ Black-box LLM

No retraining of the base model.
Works with any LLM via API.

◎ First Editing Approach

Novel editing-based method
for competitive programming.

◆ Constant Sample Budget

No large-scale sampling.
Fixed number of LLM calls.

Self-Edit Pipeline

04 / 12

Three-stage generate-and-edit workflow

01

Generate

LLM produces initial code from problem description & example test cases

02

Execute

Run code on example test case
Capture: Pass / Wrong Answer or Error with message

03

Edit

Fault-aware editor receives description + code + feedback
Outputs corrected code

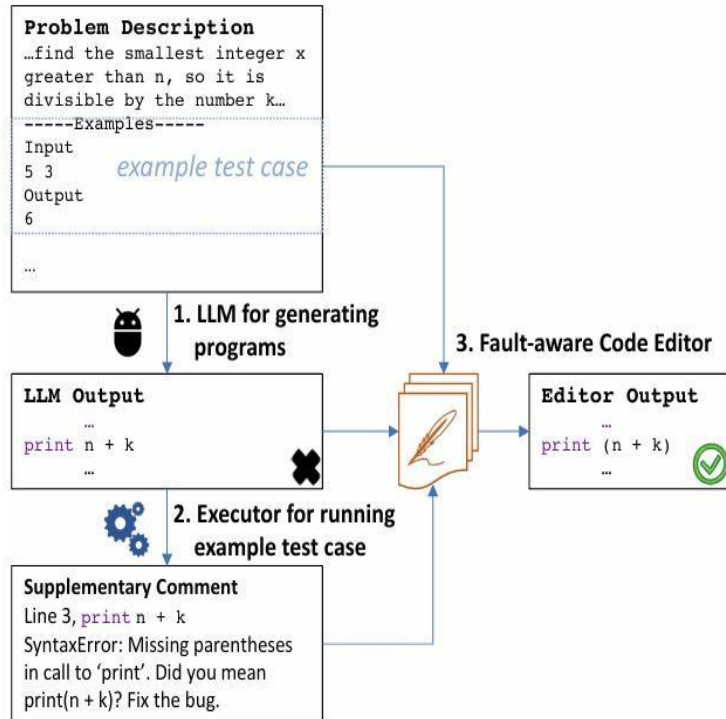


Figure 2 — Self-Edit pipeline diagram

Execution Feedback

05 / 12

Supplementary Comments — the bridge between errors and corrections

✓ Passed

"Pass the example test case."

Editor may still optimize

✗ Wrong Answer

"Expected Y, got Z. Rewrite the code."

Triggers full algorithm rewrite

⚡ Error

"Line N, <code>. <ErrorType>: msg. Fix the bug."

Points to exact line + error type

Comment 1:

Pass the example test case.

Comment 2:

Template: Wrong Answer with input: <input>. Expected output is <output_1>, but generated output is <output_2>. Rewrite the code.

Example: Wrong Answer with input: 2 5 3. Expected output is 1, but generated output is 0. Rewrite the code.

Comment 3:

Template: Line <lineno>, <line_content>, <error_msg>. Fix the bug.

Example: Line 2,
`return len([i for i in str(i**2) for i in range(n+1) if i == str(d)])`
NameError: name 'i' is not defined. Fix the bug.

Figure 4 — Supplementary Comment templates

Fault-Aware Code Editor

06 / 12

Technical design of the neural editor model

INPUT FORMAT

[SOS] description [CODE] program [CMNT] comment [EOS] → $\hat{C}$ (corrected code)

Sequence-to-Sequence

- › Input: (N, S, C) triplet
- › N = problem description
- › S = LLM-generated code
- › C = supplementary comment
- › Output: corrected code $\hat{C}$

Model: PyCodeGPT-110M

- › Decoder-only transformer
- › Fine-tuned on APPS-train
- › ~4,500 programs per LLM

Training: GOLD Objective

- › Off-policy importance weight
- › Focus on high-likelihood tokens
- › Encourages copying valid code
- › ~19% shared prefix retained

Experimental Setup

07 / 12

Datasets · Models · Metrics



APPS Dataset

Split	Problems	Avg. Tests
Train (APPS-train)	5,000	5.56
Dev (APPS-dev)	598	4.03
Test – Introductory	1,000	21.19
Test – Interview	3,000	21.19
Test – Competition	1,000	21.19

HumanEval

- › 164 programming problems
- › 8.08 avg. hidden tests
- › OpenAI benchmark
- › Out-of-distribution test

Metrics

- › pass@k — % problems solved
- › sol@k — total correct programs
- › $k \in \{1, 5, 10\}$

9 LLMs

GPT-3 (175B)

GPT-J (6B)

InCoder-1B

GPT-Neo 1.3B

CodeGen-2B

GPT-Neo 125M

CodeGen-350M

PyCodeGPT

text-davinci-002

Results

08 / 12

Self-Edit improves all 9 models across all metrics

+89%

APPS-dev
pass@1 avg.

+31%

APPS-test
pass@1 avg.

+48%

HumanEval
pass@1 avg.

GPT-3 (175B)

26.6% → 32.4% pass@1

Code Gen. Model	Para.	pass@1	edit pass@1	pass@5	edit pass@5	pass@10	edit pass@10	sol@10	edit sol@10
<i>finetuned</i>									
PyCodeGPT	110M	4.8	11.4	7.9	15.1	8.9	17.1	286	659
GPT-Neo 125M	125M	1.5	8.5	6.7	10.2	10.2	17.2	102	501
CodeGen-350M	350M	1.7	5.7	2.5	9.2	3.2	13.5	103	339
GPT-Neo 1.3B	1.3B	4.0	10.5	10.9	18.6	17.2	25.4	200	663
InCoder-1B	1.3B	9.4	12.4	12.5	16.2	13.5	18.1	568	730
GPT-J	6B	6.0	12.0	17.9	27.8	24.6	37.8	365	750
<i>zero-shot</i>									
InCoder-1B	1.3B	0.2	4.7	0.8	7.7	1.2	9.9	13	270
CodeGen-2B	2.7B	1.3	7.4	5.9	14.0	9.7	19.7	92	438
text-davinci-002	175B	26.6	32.4	43.8	48.8	49.7	58.0	1626	1948

Table 2 — APPS-dev: pass@k and sol@10 across 9 LLMs (finetuned & zero-shot)

Self-Edit vs. CodeRanker

09 / 12

Editing outperforms ranking with far fewer samples

Feature	CodeRanker	Self-Edit ✓
Approach	Reranking	Editing
Programs needed	100 per problem	1–5 per problem
pass@1 APPS-test	0.3%	0.68%
Run time	≥ 110 sec	~ 12 sec
LLM calls	Many	Few
Problems solved	Baseline	+19 more

Key Advantages

- ✓ Less compute resources
- ✓ Fewer LLM API calls
- ✓ Better pass@1 accuracy
- ✓ Works with limited budget
- ✓ More practical for real use
- ✓ Constant sample overhead

Ablation Studies

10 / 12

What matters most in the system?

Study 1 — Remove Comments

Setting	@1 dev	@5 dev	@1 test
Base model	4.8	7.9	0.2
After edit ✓	11.4	15.1	0.6
No comments	9.4	11.5	0.3

Conclusion

Execution feedback is the most critical component.
Without comments, APPS-test performance barely improves.
Losing comments ≈ losing most of the gain.

Feedback = the most important component

Study 2 — Multiple Edit Rounds

Setting	dev @1	test @1
After 1st edit ✓	11.4	0.64
After 2nd edit	11.7	0.40 ↓

Conclusion

One edit round is optimal. A second round hurts on APPS-test because:

- › Editor was trained on LLM outputs
- › At test time, it edits its own output
- › Creates a train / test distribution gap
- › Single-round editing is the right choice

One round = optimal — more hurts

Limitations & Future Work

11 / 12

Honest assessment of current boundaries and next directions

Current Limitations

-  **Small editor (110M params)**
May lack capacity for very complex bugs and deep algorithmic issues
-  **Small training dataset**
Only ~10 samples per problem (~4.5k programs total per LLM)
-  **Single-round editing only**
Multi-round iteration degrades on APPS-test distribution
-  **Compute cost gaps**
No full computational comparison against all post-processing baselines



Future Directions

-  **Larger editor models**
Experiment with CodeGen-2B+ architectures for deeper corrections
-  **Richer training data**
More diverse samples, extended coverage of problem types
-  **Multi-round editing models**
Specially designed for sequential self-correction pipelines
-  **In-context learning**
Leverage GPT-4 as both base model and editor — no training needed

Conclusion

Generate



Execute



Edit



Submit

01

Average +89% improvement on APPS-dev pass@1 across all 9 LLMs tested

02

A tiny 110M editor improved even GPT3-175B: pass@1 26.6% → 32.4%

03

Editing outperforms ranking — fewer samples, better results, less compute

04

Generalizes to out-of-distribution benchmarks (HumanEval +48%)

"Instead of generating more — generate smarter, execute, and fix."

Interactive Activity

Apply the Self-Edit framework — debug real code scenarios



Identify Errors



Write Feedback



Fix the Bug